Réseau : protocoles de transport et ICMP

Jérémy de ROCHEFORT

ENSIIE

jeremy.derochefort@gmail.com

Mardi 11 février 2020

t ki ?

- ENSIIE promo 2017
- Ancien prez ARISE/Muzzik
- (ex) Ingénieur réseau chez Scaleway/Online.net

Sommaire

1. La couche transport
   - Généralités
   - Le concept de port
2. Les protocoles UDP et TCP
   - Le protocole UDP
   - Le protocole TCP
3. Le protocole ICMP
   - Généralités sur ICMP
   - Quelques paquets ICMP courants

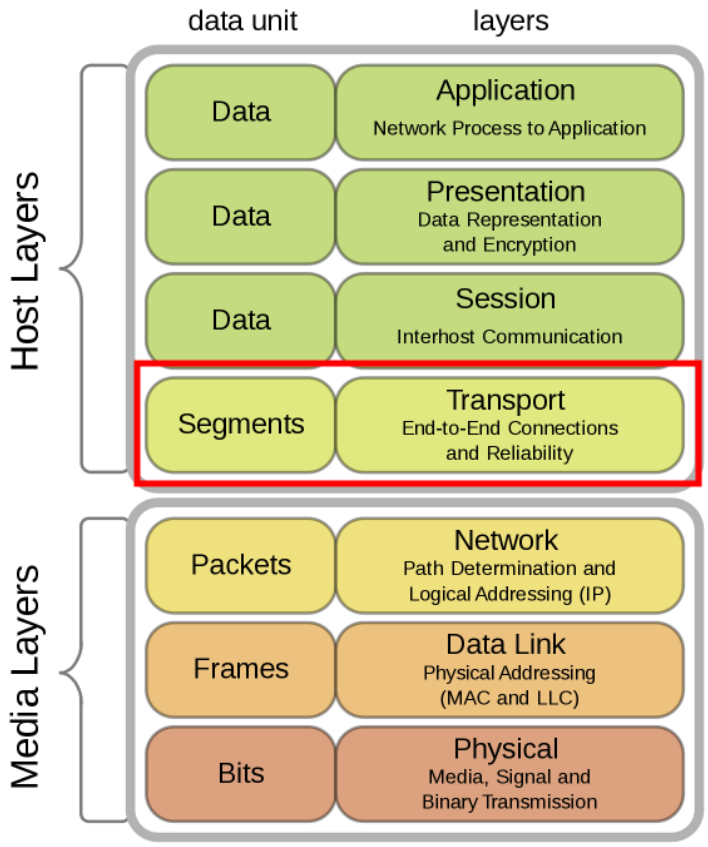
Introduction

- Protocoles de la couche 4 du modèle OSI
- Sert à transporter un payload de couche 5-7 (par exemple un paquet HTTP)
- Encapsulé dans un paquet IP
- TCP : avec connexion préalable, fiable
- UDP : "bouteille à la mer", pas de connexions, pas fiable

Acronymes et définitions

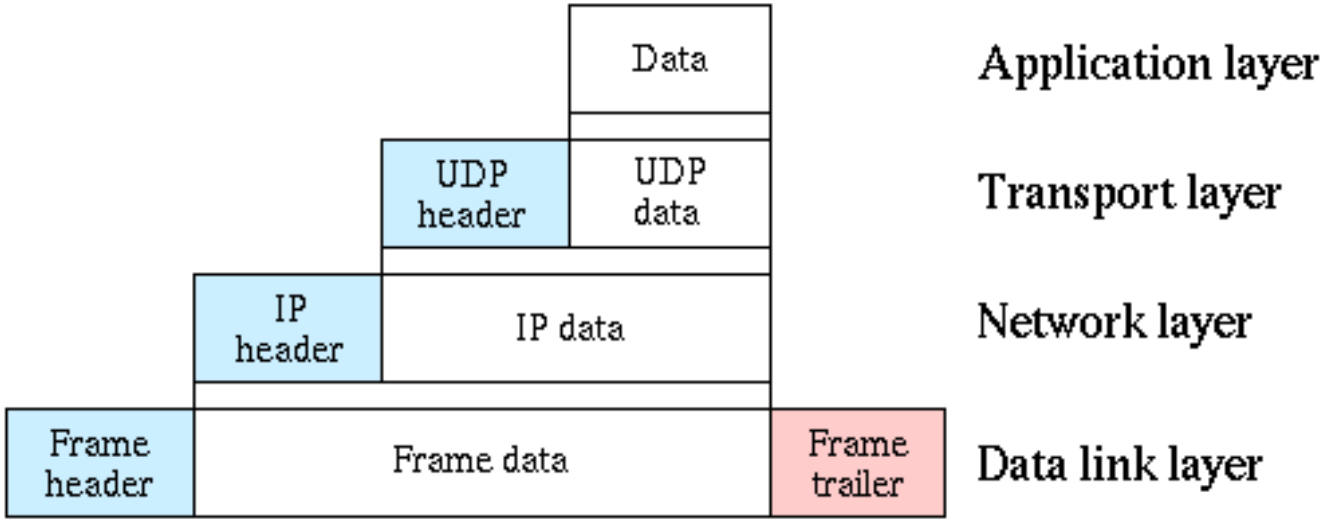
- PDU : Protocol Data Unit. aka "paquet" UDP ou TCP.
- TCP : Transmission Control Protocol
- UDP : User Datagram Protocol
- ICMP : Internet Control Message Protocol

La couche transport : lien entre réseau et application



La couche transport : lien entre réseau et applications

Illustration des encapsulations successives :



Petit arrêt au port

- Endpoint permettant à un logiciel de communiquer avec le réseau
  - Pour un serveur : port assigné au lancement du service, via lequel sont reçues les requêtes/envoyées les réponses
  - Pour un client : port assigné à l'ouverture d'un flux, utilisé pour envoyer la requête et recevoir la réponse
  - Corollaire : permet l'échange de données sans accès direct à l'OS de l'hôte

Les ports réseau : multiplexeurs/démultiplexeurs de flux

- Un PDU possède un header indiquant le port source et le port destination
- Plusieurs flux de paquets sur la même interface réseau
- Les paquets sont orientés vers les applications en fonction de leur port destination
- L'assignation d'un port au lancement d'un service permet de le différencier d'autres services sur la même machine.
- Le port et l'IP source du paquet permettent de savoir à qui envoyer la réponse

Petit arrêt au port

Une petite analogie ?

Petit arrêt au port

Une petite analogie ?

Les ports : quelques chiffres

- 65535 ports disponibles par protocole (2^{16}-1)
- Certains ports sont assignés à des protocoles spécifiques

Les ports : quelques chiffres

- 65535 ports disponibles par protocole (2^{16}-1)
- Certains ports sont assignés à des protocoles spécifiques
- Quelques exemples :
  - DNS : UDP 53
  - SSH : TCP 22
  - HTTP : TCP 80
  - HTTPS : TCP 443
- NB : Ces exemples sont des **standards** : rien n'empêche par exemple d'avoir un serveur SSH qui écoute sur le port 443.

Introduction au protocole UDP

- User Datagram Protocol
- Protocole simpliste (en comparaison de TCP)
- Pas de connexion
- Pas de garantie de fiabilité :
  - Pas de contrôle de la bonne transmission des paquets
  - Pas de contrôle de l'ordre d'arrivée des paquets

Introduction au protocole UDP

- User Datagram Protocol
- Protocole simpliste (en comparaison de TCP)
- Pas de connexion
- Pas de garantie de fiabilité :
  - Pas de contrôle de la bonne transmission des paquets
  - Pas de contrôle de l'ordre d'arrivée des paquets
- **Conséquence :** les applications doivent assurer elles-mêmes ces contrôles, ou prendre en compte leur absence (timeout, retry, etc)
- Avantage : protocole plus léger/rapide à utiliser que TCP.
- Bien adapté à des usages en temps réel, ou à de "petites" transactions. Par exemple :

Introduction au protocole UDP

- User Datagram Protocol
- Protocole simpliste (en comparaison de TCP)
- Pas de connexion
- Pas de garantie de fiabilité :
  - Pas de contrôle de la bonne transmission des paquets
  - Pas de contrôle de l'ordre d'arrivée des paquets
- **Conséquence :** les applications doivent assurer elles-mêmes ces contrôles, ou prendre en compte leur absence (timeout, retry, etc)
- Avantage : protocole plus léger/rapide à utiliser que TCP.
- Bien adapté à des usages en temps réel, ou à de "petites" transactions. Par exemple :
  - DNS
  - Voix sur IP
  - Jeux vidéo en ligne

Le header UDP

Offsets	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source port																Destination port															
4	32	Length																Checksum															

- Header simple
  - Petit (8 octets)
  - Port source
  - Port destination
  - Longueur du PDU : permet de vérifier que le paquet reçu a la bonne longueur (i.e. que les données n'ont pas été coupées)
  - Checksum : optionnel. Permet de s'assurer que le contenu a été bien transmis

UDP : l'exemple du DNS

- On lance un tcpdump
- On tape la commande `dig iiens.net` dans un terminal

UDP : l'exemple du DNS

- On lance un tcpdump
- On tape la commande `dig iiens.net` dans un terminal
- On voit ça :

```
19:57:36 jderochefort ~ » sudo tcpdump -n -i any udp port 53
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on any, link-type LINUX_SLL (Linux cooked), capture size 262144 bytes
20:00:11.260736 IP 192.168.43.145.33998 > 1.1.1.1.53: 29524+ [1au] A? iiens.net. (50)
20:00:11.264677 IP 1.1.1.1.53 > 192.168.43.145.33998: 29524 1/0/1 A 193.54.225.83 (54)
^C
2 packets captured
2 packets received by filter
0 packets dropped by kernel
```

UDP : l'exemple du DNS

- On lance un tcpdump
- On tape la commande `dig iiens.net` dans un terminal
- On voit ça :

```
19:57:36 jderochefort ~ » sudo tcpdump -n -i any udp port 53
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on any, link-type LINUX_SLL (Linux cooked), capture size 262144 bytes
20:00:11.260736 IP 192.168.43.145.33998 > 1.1.1.1.53: 29524+ [1au] A? iiens.net. (50)
20:00:11.264677 IP 1.1.1.1.53 > 192.168.43.145.33998: 29524 1/0/1 A 193.54.225.83 (54)
^C
2 packets captured
2 packets received by filter
0 packets dropped by kernel
```

- La source du paquet : `192.168.43.145.33998 >`
- Le destinataire du paquet : `> 1.1.1.1.53:`
- Le contenu de la requête

Le protocole TCP

- Protocole plus complexe qu'UDP dans son fonctionnement
- Garantie de fiabilité :
  - On garantit l'ordre de réception des paquets
  - Les paquets perdus sont réémis
- Etablissement d'une connexion avant l'envoi de données
- Acquittement (ACK/acknowledgement) de chaque byte reçu

Le protocole TCP

- Protocole plus complexe qu'UDP dans son fonctionnement
- Garantie de fiabilité :
  - On garantit l'ordre de réception des paquets
  - Les paquets perdus sont réémis
- Etablissement d'une connexion avant l'envoi de données
- Acquittement (ACK/acknowledgement) de chaque byte reçu

Conséquences :

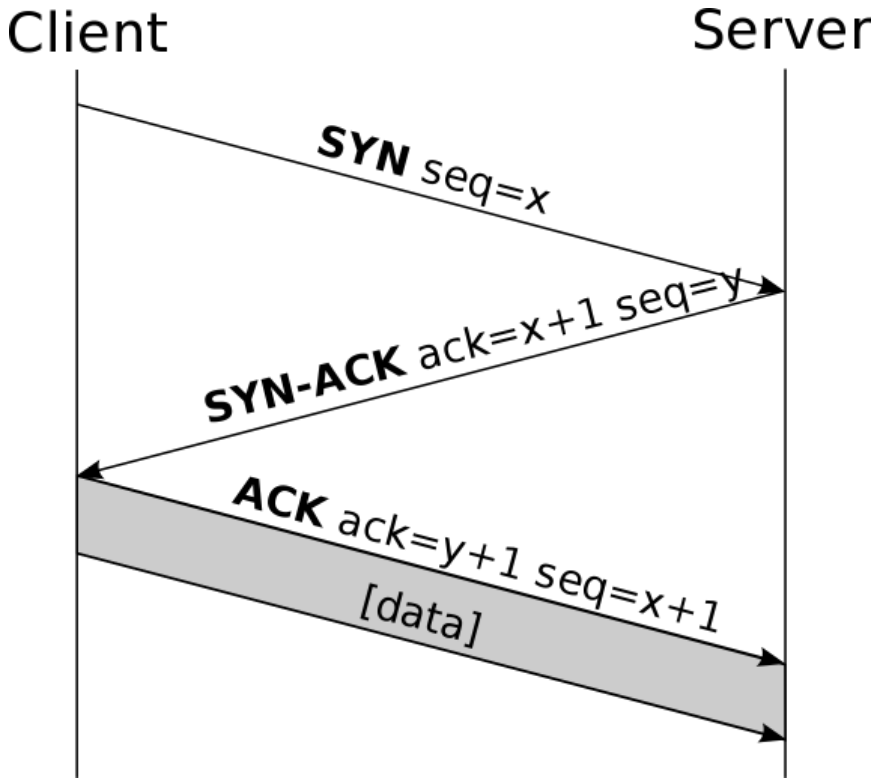
- Protocole plus lourd, overhead plus important
- Moins adapté aux échanges courts
- Le destinataire doit fonctionner correctement pour permettre l'envoi de données
- **Mais :** protocole plus fiable, gère mieux les pertes de paquets.

Le 3-way handshake de TCP

Le 3-way handshake de TCP

- Mécanisme d'établissement d'une connexion TCP
- Se déroule en 3 étapes
- Les deux parties vérifient au préalable qu'ils sont prêts à envoyer et recevoir des données
- Initialisation des deux numéros de séquence

Le 3-way handshake de TCP



Le header TCP

Offsets	Octet	0								1								2								3							
Octet	Bit	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	0	Source port																Destination port															
4	32	Sequence number																															
8	64	Acknowledgment number (if ACK set)																															
12	96	Data offset				Reserved **0 0 0**			N S	C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window Size															
16	128	Checksum																Urgent pointer (if URG set)															
20 ...	160 ...	Options (if *data offset* > 5. Padded at the end with "0" bytes if necessary.) ...																															

Le header TCP

Offsets	Octet	0								1								2								3							
Octet	Bit	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	0	Source port																Destination port															
4	32	Sequence number																															
8	64	Acknowledgment number (if ACK set)																															
12	96	Data offset				Reserved 0 0 0			N S	C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window Size															
16	128	Checksum																Urgent pointer (if URG set)															
20	160	Options (if *data offset* > 5. Padded at the end with "0" bytes if necessary.)																															
...	...	...																															

- On a toujours port source, port destination, checksum.

Le header TCP

Offsets	Octet	0								1								2								3							
Octet	Bit	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	0	Source port																Destination port															
4	32	Sequence number																															
8	64	Acknowledgment number (if ACK set)																															
12	96	Data offset				Reserved 0 0 0			N S	C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window Size															
16	128	Checksum																Urgent pointer (if URG set)															
20	160	Options (if *data offset* > 5. Padded at the end with "0" bytes if necessary.)																															
...	...	...																															

- On a toujours port source, port destination, checksum.
- Sequence number/ack number : indique le nombre de paquets transmis et reçus depuis le début de la connexion (plus de détails après).

Le header TCP

Offsets	Octet	0								1								2								3							
Octet	Bit	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	0	Source port																Destination port															
4	32	Sequence number																															
8	64	Acknowledgment number (if ACK set)																															
12	96	Data offset				Reserved 0 0 0			N S	C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window Size															
16	128	Checksum																Urgent pointer (if URG set)															
20	160	Options (if *data offset* > 5. Padded at the end with "0" bytes if necessary.)																															
...	...	...																															

- On a toujours port source, port destination, checksum.
- Sequence number/ack number : indique le nombre de paquets transmis et reçus depuis le début de la connexion (plus de détails après).
- Data offset : indique la taille du header (en multiples de 4 bytes). Vaut 5 si pas d'options utilisées.

Le header TCP

Offsets	Octet	0								1								2								3							
Octet	Bit	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	0	Source port																Destination port															
4	32	Sequence number																															
8	64	Acknowledgment number (if ACK set)																															
12	96	Data offset				Reserved 0 0 0			N S	C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window Size															
16	128	Checksum																Urgent pointer (if URG set)															
20	160	Options (if *data offset* > 5. Padded at the end with "0" bytes if necessary.)																															
...	...	...																															

- On a toujours port source, port destination, checksum.
- Sequence number/ack number : indique le nombre de paquets transmis et reçus depuis le début de la connexion (plus de détails après).
- Data offset : indique la taille du header (en multiples de 4 bytes). Vaut 5 si pas d'options utilisées.
- Des flags : on va en voire quelques uns.

Le header TCP

Offsets	Octet	0								1								2								3							
Octet	Bit	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	0	Source port																Destination port															
4	32	Sequence number																															
8	64	Acknowledgment number (if ACK set)																															
12	96	Data offset				Reserved 0 0 0			N S	C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window Size															
16	128	Checksum																Urgent pointer (if URG set)															
20	160	Options (if *data offset* > 5. Padded at the end with "0" bytes if necessary.)																															
...	...	...																															

- On a toujours port source, port destination, checksum.
- Sequence number/ack number : indique le nombre de paquets transmis et reçus depuis le début de la connexion (plus de détails après).
- Data offset : indique la taille du header (en multiples de 4 bytes). Vaut 5 si pas d'options utilisées.
- Des flags : on va en voire quelques uns.
- Urgent pointer : généralement inutilisé.

Le header TCP

Offsets	Octet	0								1								2								3							
Octet	Bit	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	0	Source port																Destination port															
4	32	Sequence number																															
8	64	Acknowledgment number (if ACK set)																															
12	96	Data offset				Reserved 0 0 0			N S	C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window Size															
16	128	Checksum																Urgent pointer (if URG set)															
20	160	Options (if *data offset* > 5. Padded at the end with "0" bytes if necessary.)																															
...	...	...																															

- On a toujours port source, port destination, checksum.
- Sequence number/ack number : indique le nombre de paquets transmis et reçus depuis le début de la connexion (plus de détails après).
- Data offset : indique la taille du header (en multiples de 4 bytes). Vaut 5 si pas d'options utilisées.
- Des flags : on va en voire quelques uns.
- Urgent pointer : généralement inutilisé.
- Options : diverses options (on va voir quelques exemples)

TCP : numéro de séquence, numéro d'acquittement

- Numéro de séquence :
  - Valeur aléatoire au début de la connexion
    - Permet d'éviter (un peu) le spoofing
    - Permet de discriminer différentes connexions successives
  - Incrémenté de 1 pour chaque byte envoyé jusqu'au paquet précédent inclus
  - NB : un paquet SYN incrémente la valeur de 1
  - Analogie : "pagination" des données envoyées

TCP : numéro de séquence, numéro d'acquittement

- Numéro de séquence :
  - Valeur aléatoire au début de la connexion
    - Permet d'éviter (un peu) le spoofing
    - Permet de discriminer différentes connexions successives
  - Incrémenté de 1 pour chaque byte envoyé jusqu'au paquet précédent inclus
  - NB : un paquet SYN incrémente la valeur de 1
  - Analogie : "pagination" des données envoyées
- Numéro d'acquittement :
  - Prend la valeur du dernier numéro de séquence reçu, s'il est cohérent
  - Représente le nombre de bytes reçus (dans le bon ordre)
  - Permet à l'émetteur de vérifier que les données sont bien reçues
  - Analogie : accusé réception

TCP : numéro de séquence, numéro d'acquittement

Time	192.168.1.2 → 174.143.213.18	Comment
0.000	(54841) SYN → (80)	Seq = 0 Ack = 94856056
0.047	(54841) ← SYN, ACK (80)	Seq = 0 Ack = 1
0.047	(54841) ACK → (80)	Seq = 1 Ack = 1
0.047	(54841) PSH, ACK - Len: 725 → (80)	Seq = 1 Ack = 1
0.097	(54841) ← ACK (80)	Seq = 1 Ack = 726
0.100	(54841) ← ACK - Len: 1448 (80)	Seq = 1 Ack = 726
0.100	(54841) ACK → (80)	Seq = 726 Ack = 1449
0.100	(54841) ← ACK - Len: 1448 (80)	Seq = 1449 Ack = 726
0.100	(54841) ACK → (80)	Seq = 726 Ack = 2897
0.100	(54841) ← ACK - Len: 1448 (80)	Seq = 2897 Ack = 726
0.100	(54841) ACK → (80)	Seq = 726 Ack = 4345
0.150	(54841) ← ACK - Len: 1448 (80)	Seq = 4345 Ack = 726
0.150	(54841) ACK → (80)	Seq = 726 Ack = 5793
0.152	(54841) ← ACK - Len: 1448 (80)	Seq = 5793 Ack = 726
0.152	(54841) ACK → (80)	Seq = 726 Ack = 7241
0.152	(54841) ← ACK - Len: 1448 (80)	Seq = 7241 Ack = 726
0.152	(54841) ACK → (80)	Seq = 726 Ack = 8689

TCP : quelques exemples de flags

- SYN : sert à l'établissement de la connexion. A pour valeur 0 après les deux premiers paquets.
- ACK : présent sur tous les paquets sauf le premier. Indique qu'il faut prendre en compte la séquence d'acquittement du paquet.
- PSH (PUSH) : envoyer les données du paquet à l'application même si le buffer n'est pas plein
  - Exemple : requête HTTP de taille inférieure à celle du buffer.
- RST (RESET) : interrompt la connexion. Les éventuels paquets reçus après sont jetés.
- FIN : dernier paquet envoyé par l'émetteur (fin de la connexion).

TCP : quelques options

- MSS : Maximum Segment Size. Indique la taille maximum de payload
  - Généralement calculé via la MTU
- SACK (Selective ACK) : permet d'acquitter des blocs de données plutôt que toutes les données séquentiellement.
- Timestamps : sert de discrmininant en cas de **wraparound** du numéro de séquence
  - Initialisé à une valeur aléatoire lors du handshake
  - Incrémenté de façon constante avec le temps
  - Deux champs :
    - sender timestamp value (timestamp de l'émetteur)
    - echo reply timestamp (timestamp la plus récente reçue du destinataire)

Le protocole ICMP

- Signifie "**I**nternet **C**ontrol **M**essage **P**rotocol"
- Protocole utilitaire : pas de payload, mais des messages d'erreurs et des information sur le réseau

Le protocole ICMP

- Signifie "**I**nternet **C**ontrol **M**essage **P**rotocol"
- Protocole utilitaire : pas de payload, mais des messages d'erreurs et des information sur le réseau
  - Le destinataire ne répond pas
  - Le TTL de paquets envoyés a expiré
  - etc... (plus de détails après)

Le protocole ICMP

- Signifie "**I**nternet **C**ontrol **M**essage **P**rotocol"
- Protocole utilitaire : pas de payload, mais des messages d'erreurs et des information sur le réseau
  - Le destinataire ne répond pas
  - Le TTL de paquets envoyés a expiré
  - etc... (plus de détails après)
- Généralement émis par les intermédiaires du réseau (routeurs)
- Encapsulé dans un paquet IP (même si c'est du L3)

Le header ICMP

Offsets	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Type								Code								Checksum															
4	32	Rest of Header																															

- Type
- Code (= "sous-type")
- Checksum
- Rest of header : informations additionnelles
  - Dépendent du type de message
  - Souvent : header + 8 premiers bytes du paquet d'origine.

Paquets ICMP : Echo request, echo reply

Echo request/Echo reply : l'outil derrière les pings

- On envoie des paquets echo requests
- On attend les reply

Paquets ICMP : Echo request, echo reply

Echo request/Echo reply : l'outil derrière les pings

- On envoie des paquets echo requests
- On attend les reply

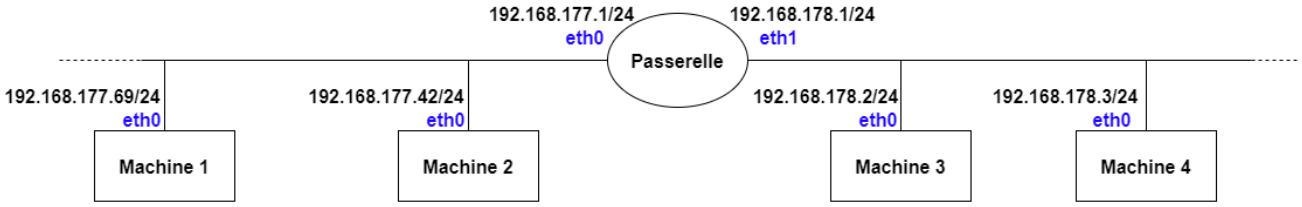
- Permet de savoir si un host (destinataire) répond
- Permet aussi de connaître le temps de réponse ($\rightarrow$ information sur la rapidité du réseau)

Paquets ICMP : Desination Unreachable

Comment savoir que le destinataire de son paquet n'est pas joignable ?

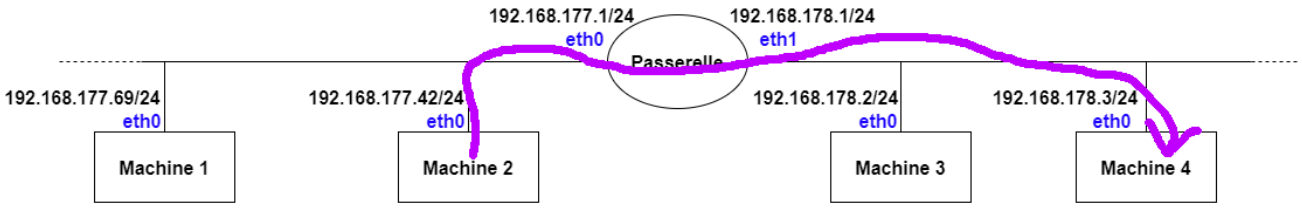
- Cas 1 : destinataire dans le même subnet
  - On ne reçoit pas de réponse aux requêtes ARP
  - $\rightarrow$ On ne peut donc pas envoyer nos paquets
- Cas 2 (le plus courant) : le destinataire est sur un réseau différent
  - Impossible de savoir a priori si l'IP destination correspond à une machine fonctionnelle
  - Les paquets sont bien envoyés au routeur
  - $\rightarrow$ On a besoin d'un moyen d'avertir l'émetteur que son paquet n'est pas arrivé

Paquets ICMP : Desination Unreachable



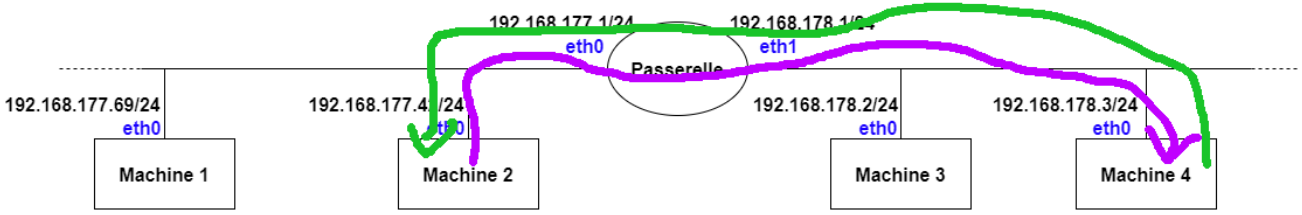
Paquets ICMP : Desination Unreachable

Dans un monde idéal :



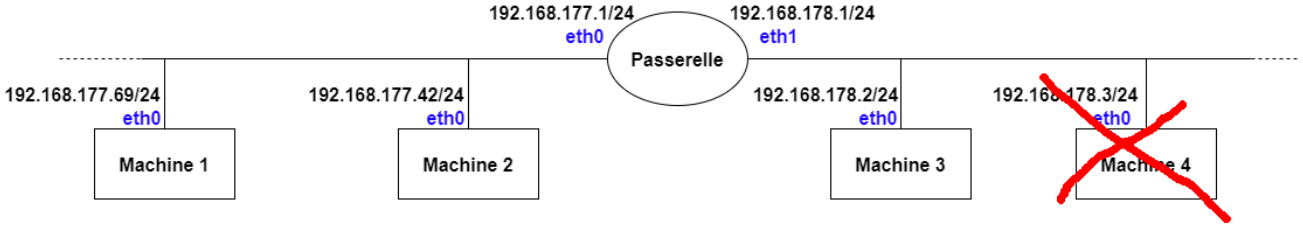
Paquets ICMP : Desination Unreachable

Dans un monde idéal :

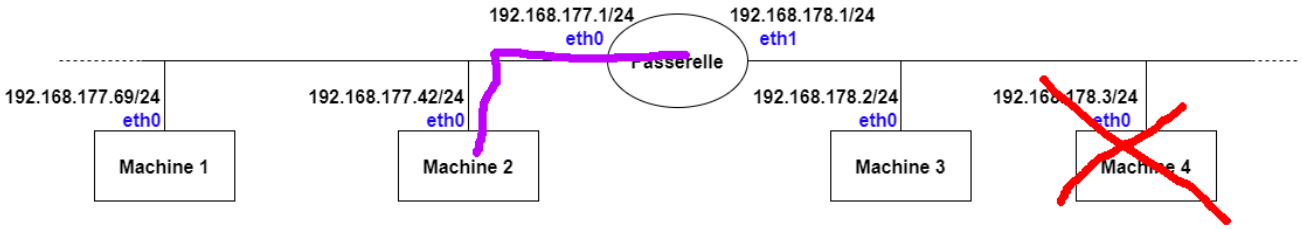


Paquets ICMP : Desination Unreachable

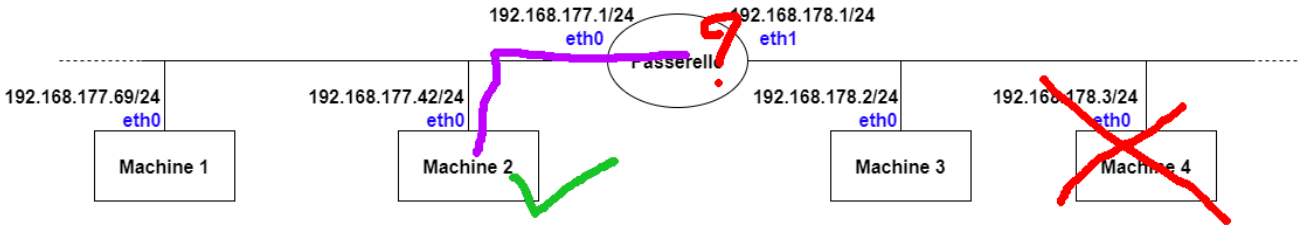
Dans un monde plein de problèmes : le destinataire est down.



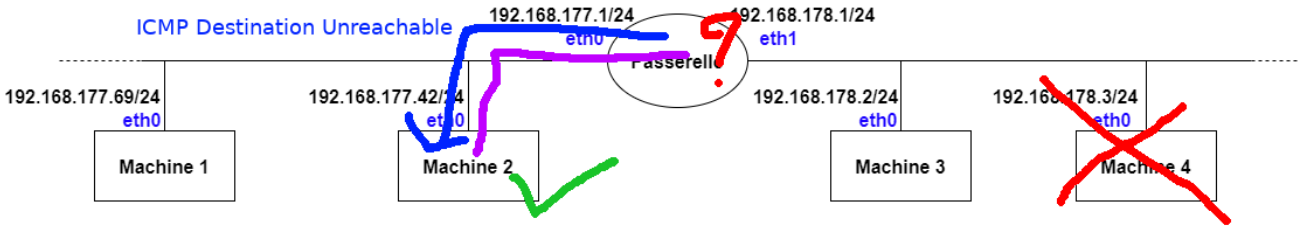
Paquets ICMP : Desination Unreachable



Paquets ICMP : Desination Unreachable



Paquets ICMP : Desination Unreachable



Paquets ICMP : Time Exceeded

- TTL (Time To Live) : mécanisme permettant d'éviter que des paquets perdus restent trop longtemps sur le réseau
- TTL décrémenté chaque fois que le paquet passe par un routeur
- Si le TTL atteint 0 après la décrémentation, le paquet est jeté
  - Paquet ICMP envoyé à l'expéditeur pour le prévenir
- Un exemple d'utilisation : traceroute
  - Envoi de paquets UDP avec des TTL de plus en plus élevés en commençant à 1
  - On liste tous les routeurs qui nous répondent

Des questions ?